

TESTs - Lesson 10 - TokenSale.sol

What is a key consideration when developing smart contracts, particularly concerning bugs?

Challenge explanation:

Smart Contract Features

Buy ERC20 Tokens



Withdraw ETH

Mint a new ERC20

Price per token

Allow users to

Update owner

A

Smart contracts cannot be easily fixed or rolled back after deployment.

B

Bugs can be easily fixed and rolled back after deployment using upgradeability patterns.

C

Testing on testnets guarantees bug-free deployment to mainnet.

D

Extensive bug bounty programs are sufficient to catch all errors.

What does ratio 'r' in the ERC20 token purchase feature of TokenSale.sol represent?

Challenge explanation:

Smart Contract Features

Buy ERC20 Tokens



Ratio per token

Withdraw ETH

Mint a new ERC20

Price per token

Allow users to

A

The current market price of ETH in USD.

B

The number of ERC20 tokens that can be bought with 1 ETH.

C

The transaction fee for buying ERC20 tokens.

D

The total supply of ERC20 tokens available for sale.

What happens to the purchased price of an NFT when a user burns it in the TokenSale.sol contract?

Smart Contract Features

Buy ERC20 Tokens

Ratio per token

Withdraw ETH

Mint a new ERC20



Price per token

Allow users to

Update owner

Allow owner to

Only burn

Architecture overview

A

The owner receives the full purchase price.

B

The user recovers half of the purchase price, and the owner can withdraw the other half.

C

The entire purchase price is permanently burned.

D

The user recovers the full purchase price.

What is the recommended approach if your smart contract needs to interact with ERC20 or ERC721 tokens but not *be* one itself?

Contracts

MyNFT.sol

MyToken.sol

TokenSale.sol

mainnet

node_modules

scripts

test

env

env.example

README



A

Inherit directly from both ERC20 and ERC721.

B

Use external calls to interact with separate token contracts.

C

Implement all token logic from scratch within a single contract.

D

Avoid interacting with other token standards for simplicity.

Why is Test-Driven Development (TDD) emphasized for smart contract development in this lesson?

- ☐ A TDD is primarily useful for creating user-friendly interfaces for dApps.
- ☒ B TDD provides a structured workflow, helps ensure correctness, gives a sense of progress, and makes debugging easier by defining expected behaviors before coding.
- ☐ C TDD is the fastest way to write smart contract code, minimizing development time.
- ☐ D TDD helps catch compiler errors but is not effective for logical bugs.

What kind of compiler error would occur if TokenSale.sol attempted to inherit directly from both ERC20 and ERC721?

- ☐ A An error related to insufficient gas fees for deployment.
- ☐ B A "not implemented" error indicating missing function definitions.
- ☒ C A "no overload matches this call" error due to conflicting function signatures and constructor ambiguity.
- ☐ D A warning about excessive code complexity but not a hard error.

What is one significant problem with using "ratios" for token conversions in Solidity smart contracts, as discussed in the video?

- ☐ A Ratios are difficult to calculate quickly on the blockchain.
- ☐ B Ratios require frequent updates from external oracles, increasing gas costs.
- ☒ C Solidity's EVM only handles integers, leading to rounding errors and potential loss of precision with floating-point numbers from ratios.
- ☐ D Ratios make smart contracts less secure and more vulnerable to attacks.

In the TokenSale.sol project, which key information needs to be passed to the constructor to link it with external ERC20 and ERC721 token contracts?

- ☐ A The total supply of each external token.
- ☐ B The names of the external token contracts.
- ☒ C The addresses of the deployed external token contracts.
- ☐ D The current market price of each external token.